

4.9 记一次 mysql 更新未成功的排查过程

作者：王向

update 更新“未成功”？

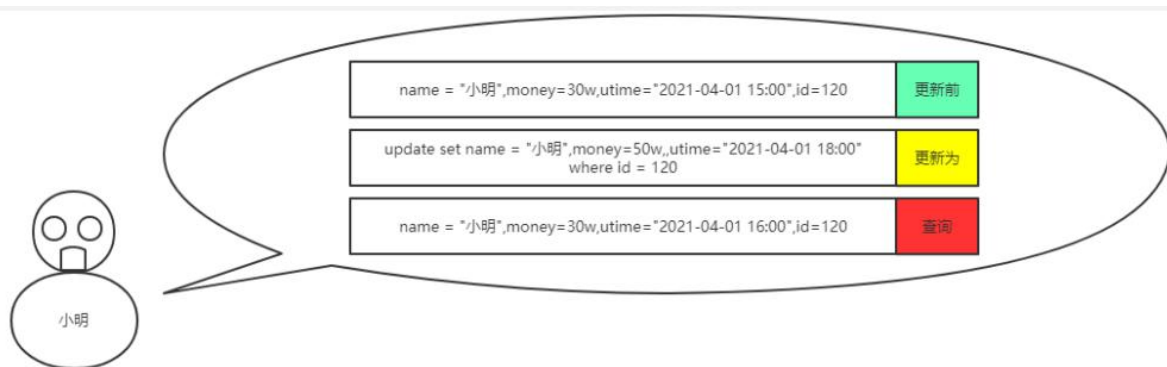
前言

笔者最近解决了一个非常曲折的问题，就是业务反映有一条数据进行 update 并且成功后，查询依然是旧数据。于是开始一路排查，最后才完美解释了所有的现象。

在这里将整个过程写成文章记录下来，希望能够对读者有所帮助。（篇幅可能会有点长，耐心看完，绝对物有所值~）

问题场景

业务小明：有一笔订单更新，更新数据返回成功，但是数据库里还是旧的数据。



看了这组数据后，百思不得其解：“不但修改的数据没有生效，utime 也不一样了”。于是登录数据库进行查询，结果的确是小明同志描述的那样。怎么办呢？

翻了一下关于这条数据的 binlog 记录的语句确实就是进行了更新，那么问题来了。这不就意味着：

写了 binlog 但并没有进行 redo-log 的更新，这不就数据不一致了？

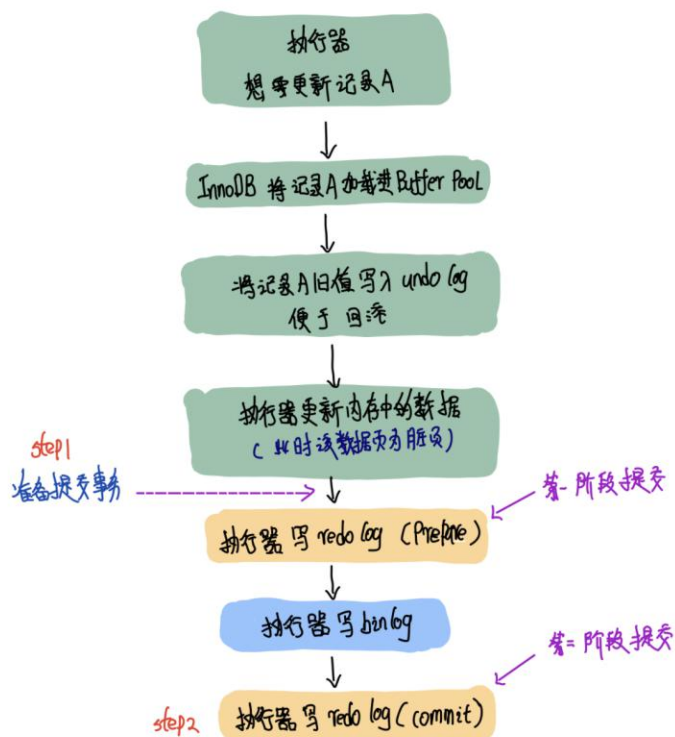
MySQL 出现“写了 binlog 但并没有写 redo-log”

众所周知，MySQL 具有 WAL 机制（先写日志，再写磁盘）。我们要搞清楚会不会出现“写了 binlog 但并没有写 redo-log”的情况，就需要研究这个 WAL 机制的二段提交特性。

在说两阶段提交事务之前，我们先来说说事务。

简单看下两阶段提交的流程

两阶段提交的时序图：



粗略观察一下上图，MySQL 想要执行事务的时候会分成两个阶段

第一阶段（prepare 阶段）：写 redo-log 并将其标记为 prepare 状态。

紧接着写 binlog，

```
#210420 16:00:20 server id 1116507691 end_log_pos 3198 CRC32 0xc306718 GTID last_committed=6 sequence_number=7
SET @@SESSION.GTID_NEXT= '3fe00d62-53d0-11eb-8eb5-02000aba4113:764514'/*!*/;
# at 3198
#210420 16:00:20 server id 1116507691 end_log_pos 3271 CRC32 0xe124fe4e Query thread_id=2 exec_time=0 error_code=0
SET TIMESTAMP=1618905620/*!*/;
BEGIN
/*!*/;
# at 3271
#210420 16:00:20 server id 1116507691 end_log_pos 3415 CRC32 0x5f37f0ff Rows_query
# INSERT INTO `t100w`.`t_100w` (`id`, `update_time`, `status`) VALUES ('161829714857801111', '2021-04-13 15:00:00', '18888')
# at 3415
#210420 16:00:20 server id 1116507691 end_log_pos 3473 CRC32 0x9d202902 Table_map: `t100w`.`t_order` mapped to number 114
# at 3473
#210420 16:00:20 server id 1116507691 end_log_pos 3537 CRC32 0x9d7f77ea Write_rows: table id 114 flags: STMT_END_F

BINLOG '
Fip+YB0rjoxCkAAAFcNAACAAHJTlNFULQgSU5UTyBgDDEwMHdgLmB0X29yZGVyYChgawRgLCBg
dXBkYXRlX3RpbWVgLCBgc3RhZHVzYkkgVtYXN0I2NzE0ODU3ODAyNTkxNicsICcy
MDIxLTA0LTEzIDE1JAw0JE1JywgJzK5Jyn/+Tdf
Fip+YBMrjoxC0gAAAJENAAAAAHIAAAAAAEABXQxMDB3AAAd0X29yZGVyYAAmPEg8FyAAAYAAEA1kg
nQ==
Fip+YB4rjoxCQAAAAENAAAAAHIAAAAAAEAAgAD//gTMTYxODI5NzE0ODU3ODAyNTkxNmpWvAP
Ajk56nd/nQ==
'/*!*/;
### INSERT INTO `t100w`.`t_order`
### SET
### @1='1618297148578025916' /* VARSTRING(200) meta=200 nullable=0 is_null=0 */
### @2='2021-04-13 15:00:15' /* DATETIME(0) meta=0 nullable=0 is_null=0 */
### @3='99' /* VARSTRING(200) meta=200 nullable=1 is_null=0 */
# at 3537
#210420 16:00:20 server id 1116507691 end_log_pos 3568 CRC32 0x6dd1da9e Xid = 111
COMMIT/*!*/;
# at 3568
```

第二阶段（commit 阶段）：写 binlog 并将其标记为 commit 状态。

```
#210420 16:00:20 server id 1116507691 end_log_pos 3198 CRC32 0x1c306718 GTID last_committed=6 sequence_number=7
SET @@SESSION.GTID_NEXT= '3fe80d62-53d0-11eb-8eb5-02000aba4113:764514'/*!*/;
# at 3198
#210420 16:00:20 server id 1116507691 end_log_pos 3271 CRC32 0xe124fe4e Query thread_id=2 exec_time=0 error_code=0
SET TIMESTAMP=1618905620/*!*/;
BEGIN
/*!*/;
# at 3271
#210420 16:00:20 server id 1116507691 end_log_pos 3415 CRC32 0x5f37f9ff Rows_query
# INSERT INTO `t100w`.`t_100w`(`id`,`update_time`,`status`) VALUES ('161829714857801111', '2021-04-13 15:00:00', '1888')
# at 3415
#210420 16:00:20 server id 1116507691 end_log_pos 3473 CRC32 0x9d202902 Table_map: `t100w`.`t_order` mapped to number 114
# at 3473
#210420 16:00:20 server id 1116507691 end_log_pos 3537 CRC32 0x9d7f77ea Write_rows: table id 114 flags: STMT_END_F

BINLOG '
Fip+YB0rjoxCkAAAAFcNAACAAHhJTLNFULQgSU5UTyBgdDEwMHdgLmB0X29yZGVyYChgaWRgLCBgc
dXBkYXRlX3RpbWVgLCBgc3RhdHVzYCKgVkfMVUVVTICgnMTYxODI5NzE0ODU3ODAyNTkxNicsICcy
MDIxLTA0LTEzIDE0JAw0JE1jywgJzk5Jyn/+Tdf
Fip+YBMrjoxC0gAAAJENAAAAHIAAAAAAEABXQxMDB3AA0X29yZGVyAAPEg8FyAAAYAAEA1kg
nQ==
Fip+YB4rjoxCQAAAAANENAAAAHIAAAAAAEAAgAD//gTMTYxODI5NzE0ODU3ODAyNTkxNmpWvVAP
Ajk56nd/nQ==
'/*!*/;
### INSERT INTO `t100w`.`t_order`
### SET
### @1='1618297148578025916' /* VARSTRING(200) meta=200 nullable=0 is_null=0 */
### @2='2021-04-13 15:00:15' /* DATETIME(0) meta=0 nullable=0 is_null=0 */
### @3='99' /* VARSTRING(200) meta=200 nullable=1 is_null=0 */
# at 3537
#210420 16:00:20 server id 1116507691 end_log_pos 3568 CRC32 0x6dd1da9e Xid = 111
COMMIT/*!*/;
# at 3568
```

两阶段写日志的意义？

你有没有想过这样一件事，binlog 默认都是不开启的！

也就是说，如果你根本不需要 binlog 带给你的特性，那你根本就用不着让 MySQL 写 binlog，也用不着什么两阶段提交。

只用一个 redolog 就够了。无论你的数据库如何 crash，redolog 中记录的内容总能让你 MySQL 内存中的数据恢复成 crash 之前的状态。

所以说，两阶段提交的主要用意是：为了保证 redolog 和 binlog 数据的安全一致性（划重点！！！拐杖敲黑板 3 次）。只有在这两个日志文件逻辑上高度一致了。你才能放心的使用 redolog 帮你将数据库中的状态恢复成 crash 之前的状态，使用 binlog 实现数据备份、恢复、以及主从复制。而两阶段提交的机制可以保证这两个日志文件的逻辑是高度一致的。没有错误、没有冲突。

排查陷入僵局

看到这里我们就发现两阶段提交保证了 redolog 和 binlog 数据的安全一致性，binlog 里进行 commit，redolog 里一定是成功的也就是说：

根本不可能出现写了 binlog 但并没有写 redo-log 的情况，完全不会出现小明描述的那样的问题。

经过反复思考，真实(しんじつ)は いつも ひとつ 平假名:しんじつはいつもひとつ（真相只有一个）。

那就是描述信息里有遗漏，在更新后和查询前必定有一个事务对这个记录进行了操作。

name = "小明",money=30w,utime="2021-04-01 15:00",id=120	更新前
update set name = "小明",money=50w,utime="2021-04-01 18:00" where id = 120	更新为
?????	必定有一个操作
name = "小明",money=30w,utime="2021-04-01 16:00",id=120	查询

排查 binlog

1. 排查这段时间内的所有和这个 id 有关的 binlog，并提取出相关记录
2. 找出更新后和查询前的那个事务的 binlog

排查这段时间内的所有和这个 id 有关的 binlog 记录

如何出排查这段时间内的所有和这个 id 有关的 binlog 记录呢，这么多的 binlog。那只能写个脚本进行批处理了。

```
file_list=$(ls mysql-bin.00*)
for i in file_list
do
    count=`mysqlbinlog -vv -d t100w.t_250w $i |grep -c "{主键 id}"`
    [ $count -gt 0 ] && (echo $i $count)
done
```

代码解释：

mysqlbinlog -d t100w.t_250w 只查看 t100w 库 t_250w 表的 binlog

grep -c 统计文件中搜索关键字的个数（等价于 select count(*) from table where id > ?）

通过 ls 获取到所有 mysql-bin，通过 for 循环找到搜索关键字的个数大于 0 的文件，并打印文件名和统计个数

```
[root@wx-super 3306]# sh 虚假的演示.sh
mysql-bin.009820 5
mysql-bin.009824 3
```

个数

然后使用 less 命令，搜索主键 id。

```
mysqlbinlog -vv -d t100w.t_250w mysql-bin.009820|less
# less 内
# /主键 id
```

最终找到了那条记录，真相大白：

name = "小明",money=30w,utime="2021-04-01 15:00",id=120	更新前
update set name = "小明",money=50w,utime="2021-04-01 18:00" where id = 120	更新为
update set name = "小明",money=30w,utime="2021-04-01 16:00" where id = 120	真相
name = "小明",money=30w,utime="2021-04-01 16:00",id=120	查询

总结

有时候问题并没有那么复杂，并不是 MySQL 底层出了问题，而是“小明”在提供信息的时候不一定能把问题准确的描述清楚。导致我们的一些误判。在处理问题上应该先从距离问题最近的角度出发。感谢大家 看完一名菜鸡 DBA 的文章。我们下个文章再见！